

STONNE: Enabling Cycle-Level Microarchitectural Simulation for DNN Inference Accelerators[†]

Francisco Muñoz-Martínez¹, José L. Abellán², Manuel E. Acacio¹ and Tushar Krishna³

Abstract—The design of specialized architectures for accelerating the inference procedure of Deep Neural Networks (DNNs) is a booming area of research nowadays. While first-generation rigid accelerator proposals used simple fixed dataflows tailored for dense DNNs, more recent architectures have argued for flexibility to efficiently support a wide variety of layer types, dimensions, and sparsity. As the complexity of these accelerators grows, the analytical models currently being used prove unable to capture execution-time subtleties, thus resulting inexact in many cases. We present STONNE (*Simulation Tool of Neural Network Engines*), a cycle-level microarchitectural simulator for state-of-the-art rigid and flexible DNN inference accelerators that can plug into any high-level DNN framework as an accelerator device, and perform full-model evaluation of both dense and sparse real, unmodified DNN models.

I. INTRODUCTION

DEEP Neural Networks (DNNs) constitute nowadays a promising breakthrough for a large number of artificial intelligence (AI) applications. The fact that their inference phase must be primarily done *in-situ* has paved the way for the development of a plethora of accelerator architectures so as to maximize performance per watt while meeting latency and energy constraints. First-generation *rigid* DNN inference accelerators (e.g. Google’s TPU [1]) focused their designs on fixed-size clusters of multiply-and-accumulate units interconnected by means of a fixed tightly-integrated on-chip network fabric specifically tailored to efficiently support a particular dataflow. This has motivated recent proposals, such as MAERI [2] and SIGMA [3], that advocate using *flexible* DNN accelerator fabrics, which can be reconfigured to efficiently map different dataflows and dot product partitions through the creation of dynamic-size clusters (i.e., a set of multipliers computing the same output) in the same hardware substrate. Of course, this flexibility comes at the cost of increased architectural complexity that urges for a more exhaustive design-space exploration for fine tuning before implementation in hardware.

Architectural simulators have been extensively used during the design process of almost any computing platform. In the case of DNN accelerators, as we explain in Section II, most architectural simulators are based on simple analytical models, which while useful, we prove that are not able to fully capture important execution-time subtleties as the level of complexity of a DNN accelerator grows. Besides, these tools do not work with real input data, and therefore, do not allow for implementation and evaluation of data-dependent optimization techniques. In contrast, detailed cycle-level simulators are capable of doing so but, to the best of our knowledge, the very few that are currently available only consider rigid DNN accelerator architectures. To bridge this gap, in this work we present STONNE (*Simulation Tool of Neural Network Engines*)¹, the first attempt to derive a cycle-level, highly-modular and highly-extensible simulator for *general* DNN inference accelerator microarchitectural exploration.

We see the following contributions in this work:

TABLE I: State-of-the-art Simulators for DNN Accelerators.

	Cycle Level	Architecture Type	Sparsity Support	FullModel Eval	DataDep Opt
MAERI BSV	✓	Flexible	✗	✗	✗
SIGMA RTL	✓	Flexible	✓	✗	✗
SCALE-Sim	✗	Rigid	✗	✗	✗
MAESTRO TimeLoop	✗	Both	✗	✗	✗
SMAUG	✓	Rigid	✗	✓	✗
STONNE	✓	Both	✓	✓	✓

- We prove the value of cycle-level simulation for accurate design-space exploration of DNN accelerators (Section II). Particularly, we show that analytical models can underestimate the runtime for certain DNN layers by more than 400%.
- We present and validate STONNE (Sections III, IV and V), the first simulator, to the best of our knowledge, that enables cycle-level microarchitectural simulation of both dense and sparse inference accelerators using mainstream PyTorch DNN models.

II. MOTIVATION AND RELATED WORK

Table I shows a qualitative comparison of STONNE with respect to contemporary publicly available tools for design-space exploration of DNN inference accelerators. For the comparison, we consider five desirable features that a DNN inference simulator should meet: 1) *cycle-level simulation*; 2) *support for both rigid and flexible DNN accelerator architectures*; 3) *support for sparse executions*; 4) *ability of performing full-model evaluations of deep learning models*; and 5) *ability to implement and evaluate data-dependent optimizations*.

Analytical Modeling. SCALE-Sim [4], MAESTRO [5] and TimeLoop [6] are recent frameworks that enable the analysis of different dataflows in DNN architectures. These tools are very powerful for fast exploration of high-level architectural details, as they are based on analytical models that calculate the degree of data reuse and computations using simple equations. These types of simulators work accurately when it comes to rigid architectures as they are simple enough to be represented by a set of formulas. However, when the complexity of the accelerator grows and/or the computation does not follow regular patterns, these models fail to faithfully capture the exact behavior of the architecture. Figure 1 shows this fact quantitatively. First, SCALE-Sim only models simple rigid architectures (e.g., TPU-like systolic arrays) and do not have support to handle sparsity. Figure 1a shows the number of cycles obtained with this analytical model and with the cycle-level execution model implemented in STONNE after running eight different representative layers (Squeeze, Expand, Factorized and Regular Convolutions, *SC*, *EC*, *FC*, *C*; Linear, *L*; and Transformers, *TR*) extracted from Squeezenet (S), Resnets-50 (R), Mobilenets (M) and BERT (B). We have configured both models to simulate an Output-Stationary systolic array varying the size of the array of processing elements (PEs) from 16×16 to 64×64 . As we can see, we obtain for the three configurations almost the same number of cycles for both alternatives, demonstrating that analytical models are valuable tools when it comes to rigid DNN accelerator architectures.

Contrarily, we have observed that analytical models fail to faithfully capture microarchitectural details of flexible DNN accelerator

¹Universidad de Murcia (Spain), ²Universidad Católica de Murcia (Spain),

³GeorgiaTech (USA). Contact author: francisco.munoz2@um.es

[†]Work supported by RTI2018-098156-B-C53 (MCIU/AEI/FEDER,UE), NSF OAC 1909900 and US Department of Energy ARIAA co-design center. F. Muñoz-Martínez supported by grant 20749/FPI/18 from Fundación Séneca.

¹The version of STONNE used to obtain the results in this letter can be found here: <https://github.com/stonne-simulator/stonne>

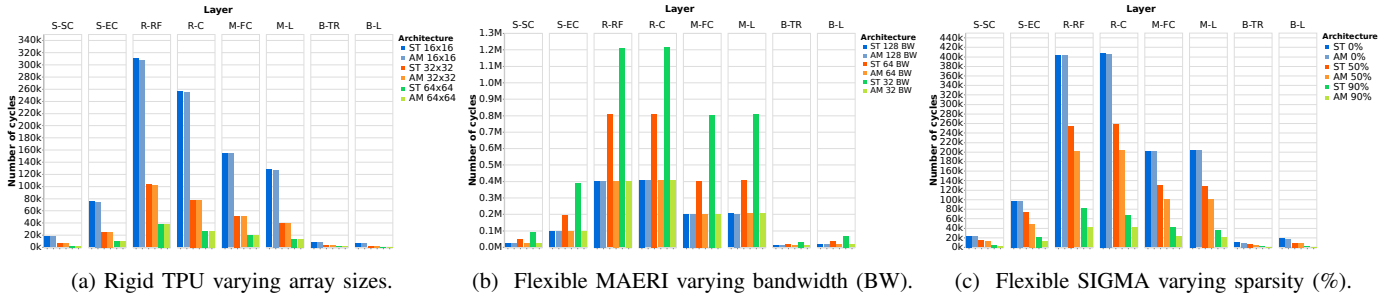


Fig. 1: Runtime for 8 DNN layers run on a simulated DNN inference accelerator using STONNE (ST) and an analytical model (AM).

architectures, and therefore, are not appropriate to identify many of the bottlenecks or unexpected behaviors that may occur during a real DNN full-model execution. To demonstrate this claim, we perform a set of experiments for a 128-multiplier flexible dense accelerator resembling MAERI [2], using the detailed analytical model provided by the authors of the MAERI paper [2]. Figure 1b plots the number of cycles reported by both STONNE and the analytical model for different global buffer bandwidth (i.e., number of elements that the global buffer can deliver per cycle to the processing elements that receive its data) configurations. In both cases we modify the parameter that controls the bandwidth to consider 128 (full bandwidth), 64 and 32 elements/cycle. We use the same layers as before. As we can see, the analytical model perfectly matches the performance obtained with STONNE when there is full bandwidth (average difference of 1.03%), as this ideal case can be easily modeled by a set of mathematical formulas. However, as the bandwidth decreases, STONNE begins to report a much higher number of cycles. This is due to the ability of a cycle-level simulator like STONNE to faithfully capture the stalls produced in the architectural pipeline and that arise as a result of increasing number of conflicts in the MAERI’s distribution and reduction networks. The difference between the results reported by STONNE and the analytical model for 32 elements/cycle increases up to 400% (see M-FC in Figure 1b), alerting about the important limitations of the analytical models.

Furthermore, we also observe that an analytical model is not capable of accurately representing DNN *sparse* executions. Figure 1c shows the same executions as before, but this time we have configured STONNE to model a sparse accelerator like SIGMA [3]. Again, we compare the results against the analytical model provided by the authors of SIGMA [3]. This time, we are assuming full bandwidth and varying sparsity ratio of the matrices from 0% to 90%. In this case, we also observe a perfect match between both STONNE and the analytical model when the sparsity ratio is 0%, but this similarity begins to diverge as the sparsity ratio increases (diverging up to 92% for a sparsity ratio of 90%). The reason for this difference is that the actual distribution of zeros in the matrices, which affects the cluster sizes, and in the end, the performance obtained by the architecture, cannot be modeled analytically. Instead, cycle-level, full-model evaluations with real weight values are needed to capture it.

Cycle-level Simulation. Among all the alternatives, only the MAERI BSV², SIGMA RTL³ implementations and SMAUG [7] claim to model flexible accelerator architectures with cycle-level precision. However, none of them really allows for efficient design-space exploration and rapid prototyping. MAERI BSV and SIGMA RTL are just two limited hardware implementations in Bluespec HDL and Verilog, respectively, written to demonstrate the effectiveness of these two flexible architectures. Hence, they are not prototypes adapted to be extended or to carry out the inference procedure of a

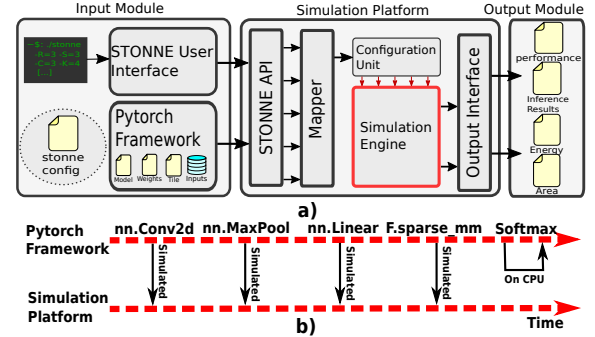


Fig. 2: a) The STONNE framework. b) DNN simulation example.

complete DNN model or perform design-space exploration. Although SMAUG is aimed to efficiently support full-model simulation of flexible architectures, actually this flexibility only means that it is able to execute any layer with any tile configuration mapping. However, the architectures currently being supported are a systolic array and the NVIDIA Deep Learning Accelerator (NVDLA), which cannot be considered flexible accelerators. Besides, since SMAUG is a trace-based simulator, it is unable to run whole modern DNN models, and therefore, has to resort to a sampling approach, which impedes its use for real full-model evaluations or examine data-dependent architectural optimizations. To address all the above shortcomings, STONNE is highly configurable and allows to create a large variety of rigid and flexible accelerators by composing a set of implemented microarchitectural modules, described in next section. Even for those custom-designed accelerators that does not match directly the modules implemented in STONNE, the tool can be considered as a framework template that facilitates its extension with low effort. Besides, STONNE is endowed with sparsity support, is capable of performing full-model evaluation of any contemporary DNN model, and as the simulator works with real data, it is suitable to evaluate data-dependent optimizations.

III. STONNE FRAMEWORK

Figure 2(a) shows a high-level view of STONNE with its three major modules for full-model simulation flow.

Simulation platform: This constitutes the principal block (see the central block in Figure 2(a)), since it includes the implementation of the simulated DNN accelerator (i.e., see Simulation Engine) whose different internal microarchitecture modules allow to simulate both rigid and flexible DNN accelerators. The simulation platform is interfaced by means of a set of coarse-grained instructions called the STONNE API (Table II). This API is the manner in which the input module (i.e., the DL framework) can interact with the simulated accelerator, configuring its simulation engine according to the user configuration file, loading layer and tile parameters, and configuring the weights and the inputs addresses in the main memory. The STONNE API can be easily extended to support new instructions.

²<https://github.com/hyoukjun/MAERI>

³<https://github.com/georgia-tech-synergy-lab/SIGMA>

TABLE II: STONNE API Instruction Set.

Instruction	Description
<i>CreateInstance</i>	Creates an instance of STONNE.
<i>ConfigureCONV</i>	Configures the accelerator to run a convolution operation.
<i>ConfigureLinear</i>	Configures the accelerator to run a fully-connected layer.
<i>ConfigureDMM</i>	Configures the accelerator to run a matrix multiplication (mm).
<i>ConfigureSpMM</i>	Configures the accelerator to run a sparse mm.
<i>ConfigureMaxPool</i>	Configures the accelerator to run a max pooling layer.
<i>ConfigureData</i>	Configure weights, inputs and outputs addresses from the CPU to the accelerator memory.
<i>RunOperation</i>	Launches the simulation

Input Module: Due to the flexibility that the STONNE API provides, the simulator can be fed easily using any of the well-known DL frameworks already available. To this end, we have modified the PyTorch DL framework⁴ (see the left block in Figure 2(a)) to connect it to the simulator and to make it able to run an instance of the Simulation Engine transparently to the user. This way, a PyTorch user just needs to select the typical *.pb* file with the weights, choose the inputs (e.g., a set of images or sentences) and briefly modify each DNN model to include the path of the hardware configuration file with the parameters of the accelerator to simulate, and the tile configuration for every layer. Furthermore, we also enable a STONNE User Interface to facilitate testing and debugging single-layer simulation and specific simulator configurations.

Output module: Once a simulation for a certain layer has been completed, this module is used for reporting simulation statistics such as performance, compute unit utilization, and activity counts of different components such as wires, FIFOs or SRAM usage (i.e., number of accesses). Besides, using these activity stats, this output module also reports the amount of energy consumed and the on-chip area required by the simulated architecture. To do so, STONNE employs a table-based area and energy model similar to Accelergy [8], computing total energy using the cycle-level activity stats for each module and a particular table with the different energy and area costs. Obviously, these statistics depend, for example, on the particular data format (e.g., fp16) utilized to represent the parameters of the DNN model. So, STONNE includes different energy and area tables that can be used. To derive these energy and area tables, we ran synthesis using Synopsys Design-Compiler, and place-and-route using Cadence Innovus on each module inside the TPU, MAERI and SIGMA RTL, obtaining the real energy and area numbers.

Figure 2(b) clarifies the interaction between the Input Module (i.e., PyTorch DL Framework) and the Simulation Platform over time (x-axis) when running the layers of a simple DNN model. As we can see, the execution is driven layer-by-layer by the DL framework (PyTorch in this case) that offloads compute-intensive layers (e.g., a convolution layer) to the simulated accelerator, and runs natively in the real CPU those layers that are not compute-intensive enough for acceleration (e.g., a SoftMax layer). Note that mapping of non matrix multiplication layers (i.e., layers for which other architectures require SIMD specific modules), such as Pooling (e.g., *nn.MaxPool*) and batch normalization, can be easily supported in flexible accelerator architectures [2], and thus, in STONNE. We provide functional simulation for CPU-mapped layers, and both functional and timing simulation for STONNE-mapped layers. This allows us to simulate complex DNN models such as BERT in 49 hours.

IV. STONNE SIMULATION ENGINE

STONNE implements all the major components required to build both rigid and flexible DNN accelerators (see Figure 3a).

A. On-Chip Networks

All the on-chip components are interconnected by using a general three-tier network composed of a Distribution Network (DN), a Multiplier Network (MN), and a Reduce Network (RN), inspired

by the taxonomy of on-chip communication flows within DNN accelerators [2]. These networks can be configured to support any topology to model state-of-the-art accelerators such as the TPU, Eyeriss-v2, ShDianNao, SCNN, MAERI and SIGMA, among others.

1) Distribution Networks (DNs): To deliver the single-cycle data delivery from the Global Buffer (GB) to the MN, we implement a *Tree Network (TN)* (Figure 3b(e)), a *Benes Network (BN)* (Figure 3b(f)) and a *Point-to-Point Network (PoPN)* (Figure 3b(g)). Each node in this network is a packet-switched, low-cost, bufferless Distribution Switch (DS) that selects the input destination based on a bit vector that is set by the memory controller.

2) Multiplier Networks (MNs): These networks are made up of a set of Multiplier Switches (MSs) that can be configured to act as either forwarders or multipliers. We support two MN topologies: *Linear Multiplier Network (LMN)* (Figure 3b(h)) used for data reuse and a *Disabled Multiplier Network (DMN)* (Figure 3b(i)).

3) Reduction Networks (RNs): These networks are composed of configurable packet-switched adders aimed at accumulating the different clusters of partial sums that are generated by the MN. In this case we support a *Reduction Tree (RT)*, an *Augmented Reduction Tree (ART+DIST)* (Figure 3b(a)) an *ART + Accumulation Buffer (ART+ACC)* (Figure 3b(b)) and a *Forwarding Augmented Network (FAN)* (Figure 3b(c)). In addition, we also implement a *Linear Reduction Network (LRN)* (Figure 3b(d)) used in the TPU [1].

B. Memory Hierarchy and Memory Controllers

STONNE implements the typical configurable memory hierarchy found in most DNN accelerators composed of local storage, some on-chip global storage (i.e., the Global Buffer-GB), and the off-chip DRAM memory. These three levels of the hierarchy are configurable by the user through the STONNE configuration file, which defines parameters such as bandwidth, different FIFO sizes, size of the GB and size and technology of the DRAM (e.g., HBM). Data orchestration between the GB and the distribution and reduction networks is performed by a memory controller (i.e., control unit) which is also selected by the user based on their preferences. As data movement differs depending on both the dataflow and whether the execution is dense or sparse, STONNE implements different types of memory controllers (Figures 3b(j) and 3b(k)) which are configurable and interact with DRAM memory assuming double-buffering prefetching at the Global Buffer. These memory controllers use internal counters to calculate the next addresses that the accelerator will read or write. Obviously, the configured memory controller must always be compatible with the hardware substrate selected to be modelled. In terms of dataflows, STONNE implements the weight-stationary, output-stationary and input-stationary dataflows. Regarding sparsity, the tool is able to understand both bitmap and CSR representations. Other alternatives could also be easily implemented from the existing memory controllers. DRAM is modeled using DRAMsimv3 [9].

C. Modeling DNN Accelerators in STONNE

Variability: Using the building blocks shown in Figure 3b(a-k), STONNE is able to model a variety of DNN accelerator architectures. Examples of particular architectures directly supported in STONNE and the basic building blocks used in each case are drawn in Figure 3b(l-o). Moreover, STONNE can be easily extended with additional models of DNs, MNs, RNs and memory controllers.

Data-dependent optimizations: Since STONNE connects with DL frameworks the aforementioned building blocks can be extended to precisely evaluate data-dependent architectural optimizations.

Limitations of STONNE: In essence, STONNE is able to model MAC-based accelerators. Modelling other types of accelerators (e.g., bit-wise or analog ones) could require major changes to the Simulation Platform component of STONNE.

⁴Other DL frameworks, such as Tensorflow, can be easily integrated with STONNE.

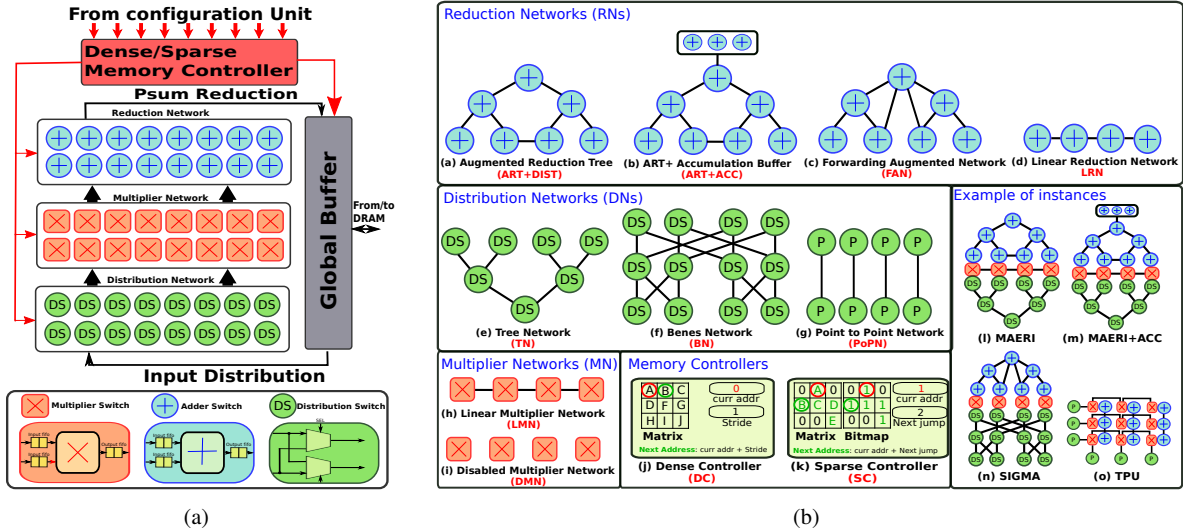


Fig. 3: (a) Overview of the general flexible DNN inference accelerator considered in STONNE. (b) Basic building blocks.

TABLE III: Timing accuracy of STONNE using RTL versions of MAERI, SIGMA and an OS-dataflow TPU.

Design	Layer	M	N	K	RTL # cycles	STONNE # cycles	Error %
MAERI	MAERI-1	6	25	54	1338	1381	3.10%
	MAERI-2	20	25	180	16120	16081	0.24%
	MAERI-3	6	400	54	26178	26581	1.51%
SIGMA	SIGMA-1	64	128	32	2321	2304	0.73%
	SIGMA-2	256	64	64	8594	8448	1.72%
	SIGMA-3	256	128	64	17192	16896	1.75%
	SIGMA-4	128	1	64	139	138	0.72%
TPU	TPU-1	16	16	32	66	67	1.50%
	TPU-2	16	16	16	50	51	2.00%
	TPU-3	32	32	16	200	204	2.00%
	TPU-4	64	64	32	1056	1072	1.50%

V. VALIDATION AND RESULTS

Timing validation: To validate the timing accuracy of STONNE against real hardware, we focus on three open-source implementations of DNN accelerators: the MAERI BSV code, the SIGMA Verilog code, and the TPU RTL implementation used to validate SCALE-Sim [4]. We configure three instances of a MAERI-like, SIGMA-like and output stationary TPU-like architecture using their corresponding building blocks (see Figures 3b(l), 3b(n) and 3b(o)). Table III shows a comparison of the total number of executed cycles reported by the RTL versions and STONNE after running the eleven layers supported by the RTL versions. As we can see, the differences in the total number of executed cycles obtained with STONNE and the RTL versions range from 0.14% to 3.10% (1.53% on average), demonstrating that STONNE closely mimics the characteristics of the hardware versions. Furthermore, we carried out timing validation across the modeled DNN accelerators (see Fig. 4). We observed that a MAERI-like architecture exhibits an average performance improvement of 20% over the TPU-like architecture for the execution of seven DNN models, with a maximum of 231% for Mobilenets and a minimum of 9% for Resnets-50. Besides, we found that a SIGMA-like architecture is 91% faster on average than a MAERI-like one thanks to the sparsity support. In terms of energy, STONNE finds that the SIGMA-like architecture is 70% and 54% more efficient across the seven DNN models than the MAERI-like and TPU-like architectures, respectively. Finally, in terms of area, the SIGMA-like architecture is 13% more efficient than the MAERI-like one. These results are inline with trends reported in prior works [2], [3].

Functional validation: To validate the functionality of STONNE, we have configured and run STONNE with 256 processing elements. We have executed the seven DNN models listed in Figure 4 with a

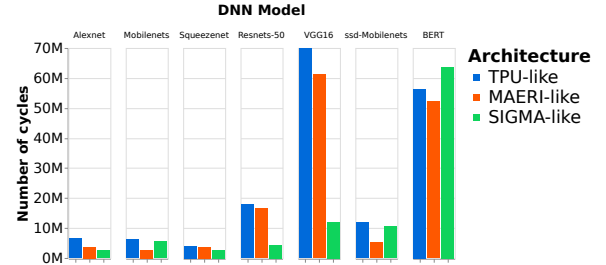


Fig. 4: Number of cycles for the 7 DNN models with STONNE.

test set of 50 samples, and for each sample, we have compared the output of the last DNN layer reported by PyTorch when run natively on the CPU, with that obtained for runs with STONNE. We have found that they fully match for all cases.

VI. CONCLUSIONS

We demonstrate that as the complexity of the microarchitecture of DNN accelerators grows, the analytical models typically used to estimate their performance and energy figures are not able to capture many important subtleties that simulation at cycle level does. STONNE aims to fill this gap, paving the way towards rapid and accurate prototyping of next-generation DNN accelerator architectures.

REFERENCES

- [1] N. P. Jouppi *et al.*, “In-datacenter performance analysis of a tensor processing unit,” in *44th Int’l Symp. on Computer Architecture*, 2017.
- [2] H. Kwon, A. Samajdar, and T. Krishna, “MAERI: Enabling flexible dataflow mapping over dnn accelerators via reconfigurable interconnects,” *Int’l Conf. on Architectural Support for Programming Languages and Operating Systems*, Mar. 2018.
- [3] E. Q. *et al.*, “SIGMA: A sparse and irregular gemm accelerator with flexible interconnects for dnn training,” *Int’l Symp. on High-Performance Computer Architecture*, Mar. 2020.
- [4] A. S. *et al.*, “SCALE-Sim: Systolic cnn accelerator simulator,” *arXiv preprint arXiv: 1811.02883v1* (2019), Feb. 2019.
- [5] H. Kwon *et al.*, “Understanding reuse, performance, and hardware cost of dnn dataflow: A data-centric approach,” *Int’l Symp. on Microarchitecture (MICRO)*, Oct. 2019.
- [6] A. P. *et al.*, “Timeloop: A systematic approach to dnn accelerator evaluation,” *Int’l Symp. on Perf. Analysis of Sys. and Soft.*, Mar. 2019.
- [7] S. Xi *et al.*, “SMAUG: End-to-end full-stack simulation infrastructure for deep learning workloads,” *ACM Trans. Archit. Code Optim.*, vol. 17, no. 4, Nov. 2020.
- [8] Y. N. Wu, J. S. Emer, and V. Sze, “Accelergy: An architecture-level energy estimation methodology for accelerator designs,” *International Conference On Computer Aided Design (ICCAD)*, Nov. 2019.
- [9] S. L. *et al.*, “DRAMsim3: A cycle-accurate, thermal-capable dram simulator,” *IEEE Computer Architecture Letters*, vol. 19, no. 2, 2020.